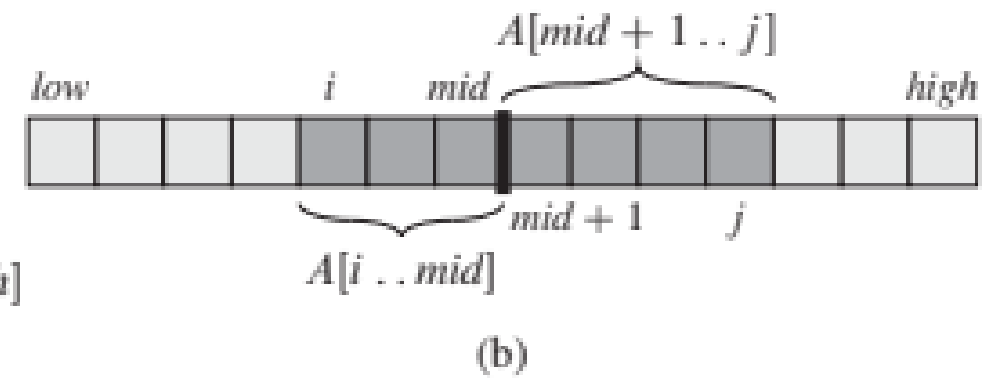
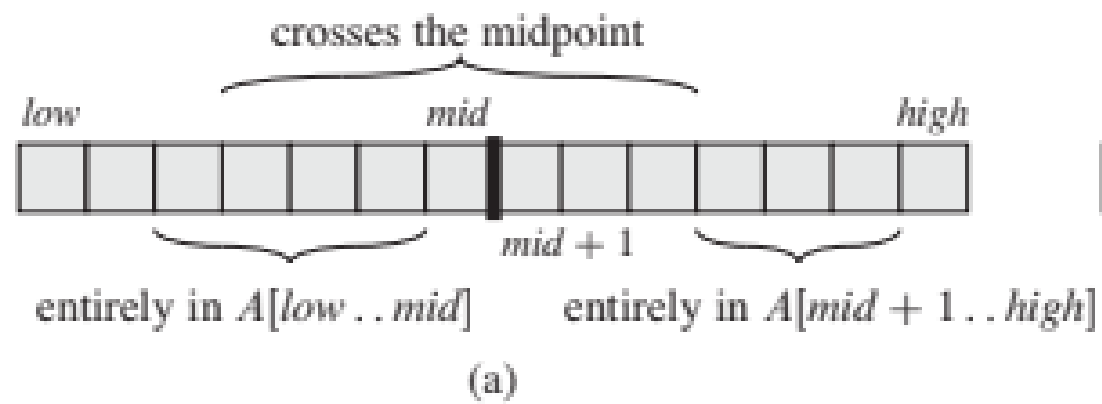


Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 10**



FIND-MAXIMUM-SUBARRAY(*A*, *low*, *high*)

```
1  if high == low
2      return (low, high, A[low])           // base case: only one element
3  else mid =  $\lfloor (\textit{low} + \textit{high}) / 2 \rfloor$ 
4      (left-low, left-high, left-sum) =
          FIND-MAXIMUM-SUBARRAY(A, low, mid)
5      (right-low, right-high, right-sum) =
          FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
6      (cross-low, cross-high, cross-sum) =
          FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
7      if left-sum  $\geq$  right-sum and left-sum  $\geq$  cross-sum
8          return (left-low, left-high, left-sum)
9      elseif right-sum  $\geq$  left-sum and right-sum  $\geq$  cross-sum
10         return (right-low, right-high, right-sum)
11     else return (cross-low, cross-high, cross-sum)
```

Example:-

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 \\ 13 & -3 & -25 & 20 & -3 & -16 & -23 & 18 & 20 & -7 & 12 & -5 & -22 & 15 & -4 & 7 \end{bmatrix} \quad ①$$

mid = 8

= FIND-MAXIMUM-SUBARRAY (A, 1, 8)

$$A' = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 13 & -3 & -25 & 20 & -3 & -16 & -23 & 18 \end{bmatrix}$$

mid = 4

= FIND-MAXIMUM-SUBARRAY (A', 1, 4)

$$A'_1 = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 13 & -3 & -25 & 20 \end{bmatrix}$$

mid = 2

$$A'_{11} = \begin{bmatrix} 1 & 2 \\ 13 & -3 \end{bmatrix}$$

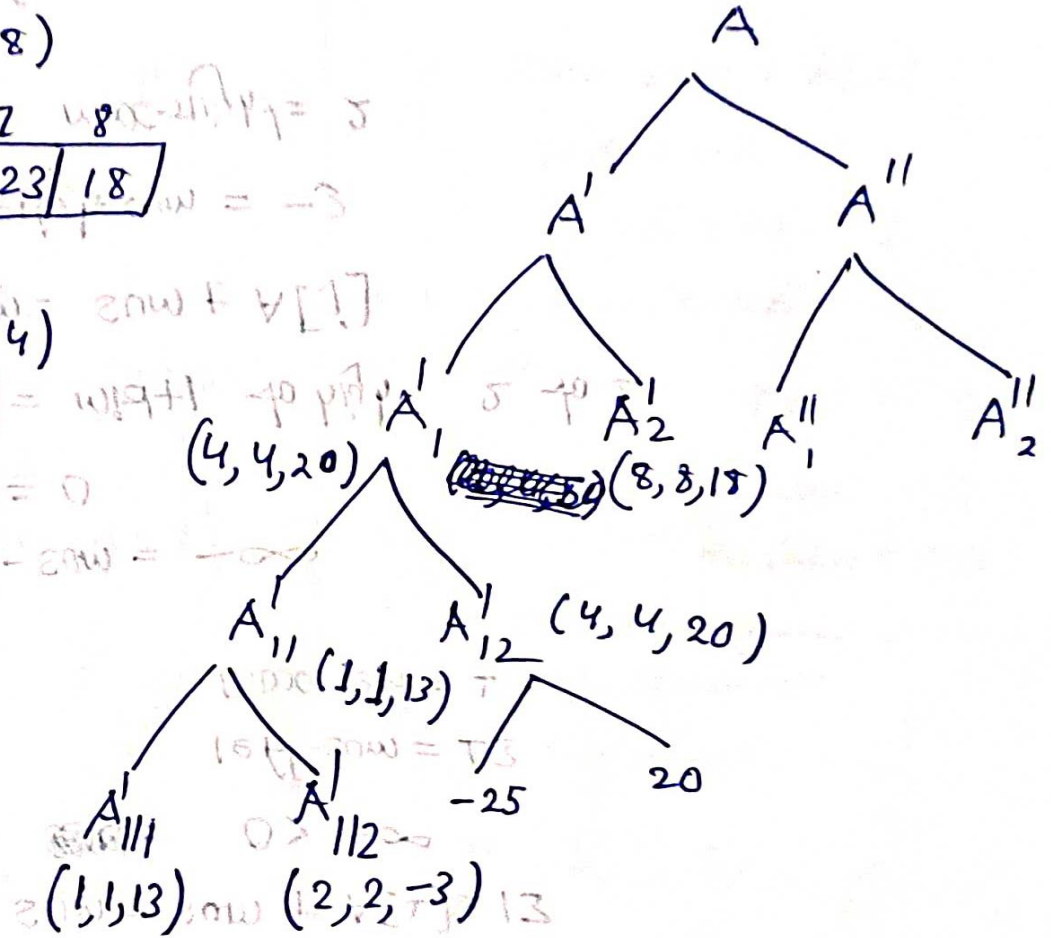
mid = 1

$A'_{111} = [13]$ Here, $\boxed{\text{high} = \text{low}}$

return (low, high, A(low))

= (1, 1, 13)

(left low, left high, left sum)



→ FIND-MAX-CROSSING-SUBARRAY (A', low, mid, high)

$$\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ [13 & -3 & -25 & 20 & -3 & -16 & -23 & 18] \end{matrix} \quad \text{([1, 8, 51])}^*$$

$$\begin{matrix} 1 & 2 & 3 & 4 \\ \text{([1, 4, 33])}^* & [13 & -3 & -25 & 20] \end{matrix} \quad \begin{matrix} 5 & 6 & 7 & 8 \\ [-3 & -16 & -23 & 18] \end{matrix} \quad \text{([8, 8, 18])}^*$$

$$\begin{matrix} 1 & 2 \\ \text{([1, 1, 13])}^* & [13 & -3] \end{matrix} \quad \begin{matrix} 3 & 4 \\ [-25 & 20] \end{matrix} \quad \text{([4, 4, 20])}^* \quad \begin{matrix} 5 & 6 \\ [-3 & -16] \end{matrix} \quad \begin{matrix} 7 & 8 \\ [-23 & 18] \end{matrix} \quad \text{([5, 5, -3])}^* \quad \text{([8, 8, 18])}^*$$

$$\begin{matrix} 1 & 2 \\ [13] & [-3] \end{matrix} \quad \begin{matrix} 3 & 4 \\ [-25] & [20] \end{matrix} \quad \begin{matrix} 5 & 6 \\ [-3] & [-16] \end{matrix} \quad \begin{matrix} 7 & 8 \\ [-23] & [18] \end{matrix}$$

- ⑤
- > The call to FIND-MAX-CROSSING-SUBARRAY in line 6 takes $\Theta(n)$ times
 - > Lines 7-11 take only $\Theta(1)$ time. For the recursive case, therefore, we have

$$\begin{aligned} T(n) &= \Theta(1) + 2T(n/2) + \Theta(n) + \Theta(1) \\ &= 2T(n/2) + \Theta(n) \end{aligned}$$

> Combining above equation:-

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2T(n/2) + \Theta(n) & \text{if } n>1 \end{cases}$$

> This recurrence is the same as recurrence for merge sort.

$$\boxed{T(n) = \Theta(n \log n)}$$

Dynamic programming
algorithm

* Kadane's algorithm steps:

> Kadane's algorithm is a dynamic programming technique that optimizes the solution to $O(n)$ time complexity.

Algorithm:-

- ① Initialize two variables, currentSum and maxSum, on the first index of the array.
- ② Iterate through the array:
 - Starting from the 2nd position, iterate through the array.
 - For each element perform
 - { $\text{currentSum} \leftarrow \max\{\text{current element}, (\text{current element} + \text{currentSum})\}$
 - Update $\text{maxSum} \leftarrow \max\{\text{currentSum}, \text{maxSum}\}$
- ③ Result: The maxSum at the end is the maximum sum of a contiguous subarray.

$$\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ [-2, & 1, & -3, & 4, & -1, & 2, & 1, & -5, & 4] \end{matrix}$$

$$CS = -2, MS = -2$$

Loop 2 to $n = \{8, 1\}$ $l=2$ $ce = 1$

$$\{ CS = \max\{ce, (CS + ce)\}$$

$$CS = \max\{1, (-2 + 1)\} = 1$$

$$MS = \max\{1, -2\} = 1$$

$$\} MS = \max\{CS, MS\}$$

$l=3$ $ce = -3$

$$CS = \max\{-3, (1 + (-3))\} = -2$$

$$MS = \max\{-2, 1\} = 1$$

$l=4$ $ce = 4$

$$CS = \max\{4, (-2 + 4)\} = 4$$

$$MS = \max\{4, 1\} = 4$$

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹
 $[-2, 1, 3, -4, -1, 2, 1, -5, 4]$

$$\underline{\underline{i=5}}$$

$$ce = -1$$

Loop

$$\left\{ \begin{array}{l} cs = \max \{ ce, (ce + cs) \} \\ ms = \max \{ ms, cs \} \end{array} \right\}$$

$$cs = \max \{ -1, 3 \} = 3$$

$$ms = \max \{ 4, 3 \} = 4$$

$$\underline{\underline{i=6}}$$

$$ce = 2$$

$$cs = \max \{ 2, 3+2 \} = 5$$

$$ms = \max \{ 4, 5 \} = 5$$

$$\underline{\underline{i=7}}$$

$$ce = 1$$

$$cs = \max \{ 1, 5+1 \} = 6$$

$$ms = \max \{ cs, ms \} = 6$$

1 2 3 4 5 6 7 8 9
[-2, 1, 3, -4, -1, 2, 1, -5, 4]

loop

{ $CS = \max\{ce, (ce + cs)\}$

$MS = \max\{MS, cs\}$

}

$i=8$

$$ce = -5$$

$$CS = \max\{-5, (-5+6)\} = 1$$

$$MS = \max\{1, 6\} = 6$$

$i=9$

$$ce = 4$$

$$CS = \max\{4, (4+1)\} = 5$$

$$MS = \max\{5, 6\} = 6$$

* Matrix multiplication:-

8

- > Given two matrices, $A(n \times m)$ and $B(m \times p)$, the result is a new matrix, $C(n \times p)$.
 - > The standard matrix multiplication algorithm computes each element of C by performing the dot product of a row from A and a column from B .
 - > This process involves multiple multiplications and additions.
 - > For instance, in case where $n=m=p=2$, a total 8 multiplication are needed.
- ⇒ Strassen's Matrix Multiplication Description:

- > Is an algorithm used to efficiently multiply two matrices
- > Developed by Volker Strassen in 1963 and is known for its ability to reduce the number of arithmetic operations required to multiply two matrices.

Basic Matrix Multiplication

```
void matrix_mult (){  
    for (i = 1; i <= N; i++) {  
        for (j = 1; j <= N; j++) {  
            compute Ci,j;  
        }  
    }  
}
```

algorithm

Time analysis

$$C_{i,j} = \sum_{k=1}^N a_{i,k} b_{k,j}$$

$$\text{Thus } T(N) = \sum_{i=1}^N \sum_{j=1}^N \sum_{k=1}^N c = cN^3 = O(N^3)$$

Strassen Matrix Multiply :

(10)

① # split A and B of size $n \times n$, where n is a power of 2.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \text{ and } B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

② # Calculate seven products recursively using these submatrices

$$P_1 = A_{11} \times (B_{12} - B_{22})$$

$$P_2 = (A_{11} + A_{12}) \times B_{22}$$

$$P_3 = (A_{21} + A_{22}) \times B_{11}$$

$$P_4 = A_{22} \times (B_{21} - B_{11})$$

$$P_5 = (A_{11} + A_{22}) \times (B_{11} - B_{22})$$

$$P_6 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$P_7 = (A_{11} - A_{21}) \times (B_{11} + B_{12})$$

③ # compute the resulting submatrices

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

④ # Combine the submatrices into the result matrix C

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Time Complexity Analysis:

- > The recurrence equation for Strassen's Matrix Multiplication can be expressed as follows:

$$T(n) = 7T(n/2) + O(n^2)$$

- > The term $7T(n/2)$ accounts for the seven recursive multiplications performed on smaller submatrices, and $O(n^2)$ represents the time needed for additional operations.

- > Using "Master Theorem" $\rightarrow$ we will obtain $O(n^{2.81})$

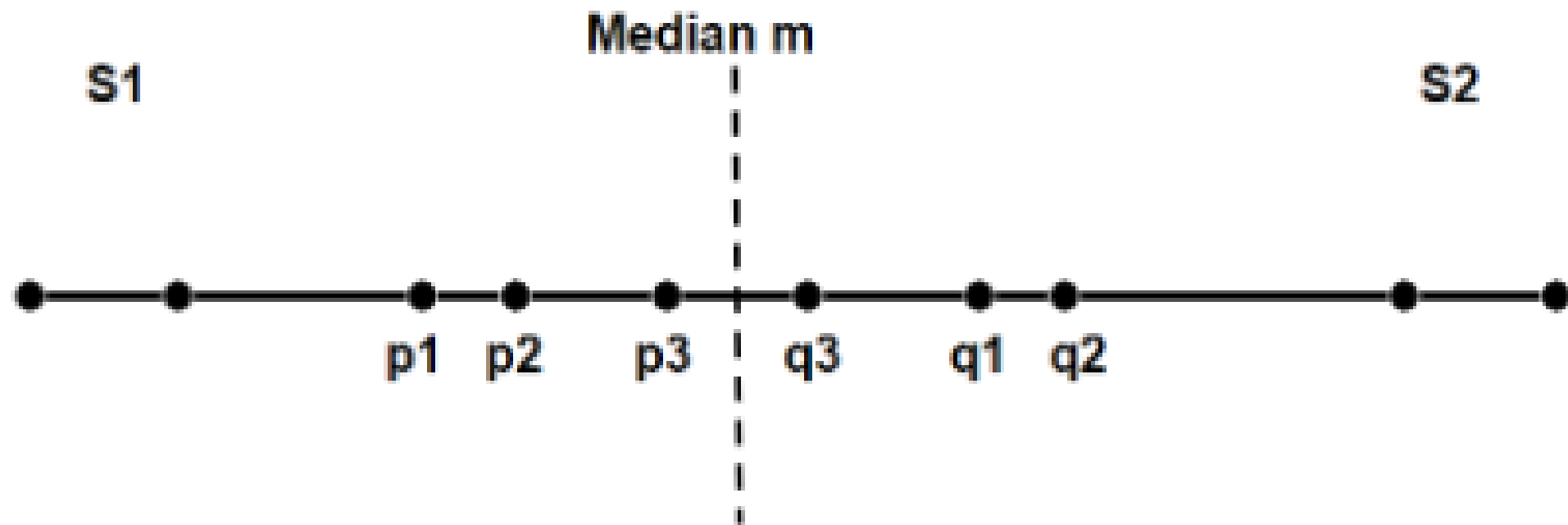
Closest pair problem

- The Closest pair problem is a fundamental computational challenge that revolves around the task of identifying the pair of points with the smallest distance between them within a given collection of N points.
- This problem finds relevance and application across various domains, from computational geometry to pattern recognition and beyond. The choice of the most suitable algorithm to tackle this problem depends on two crucial factors: the specific value of N , representing the number of points in the set, and the desired outcome, which could involve finding the actual distance, the point coordinates, or both.
- As we navigate through this problem, we encounter a spectrum of scenarios and computational intricacies, all of which stem from the variable N and the specific requirements of the task at hand. The larger the N , the more complex the problem becomes, and the choice of algorithm becomes critical to achieve efficient solutions.

- In the realm of brute-force approaches, it is evident that the Closest Pair Problem can be solved in $O(N^2)$ time.
- This section delves into the intricacies of a divide-and-conquer-based algorithm that has been specifically crafted to address the Closest Pair Problem in a more efficient and effective manner.
- This algorithm optimizes the search for the closest pair of points, significantly reducing the time complexity, especially when N is substantial.

Divide-and-conquer algorithm for solving closest pair problem:

- The central concept involves divide-and-conquer based algorithms for solving closest pair problems by dividing the set of points in half and recursively searching for the closest pair of points within each half.
- To comprehend the solution approach more thoroughly, we'll explore it from three different perspectives: the 1-dimensional (1D), 2-dimensional (2D), and d-dimensional (dD) closest pair problem.



1D_CLOSEST_PAIR(N)

1. **if** the size of set N is 1, the output is $\delta = \infty$
2. **if** the size of set N is 2, the output is $\delta = |p_2 - p_1|$
3. **for** larger sets:
 m = Compute the median of x -coordinates
 Divide N into two subsets, N_1 and N_2 , at m .

Compute δ_1 as *1D_CLOSEST_PAIR*(N_1)

Compute δ_2 as *1D_CLOSEST_PAIR*(N_2)

Determine δ_{12} as the minimum distance across the cut.

return $\delta = \min(\delta_1, \delta_2, \delta_{12})$ and corresponding pair

- The recurrence equation for the 1D closest pair problem can be expressed as follows $T(n)=2T(n/2)+O(n)$.
- In this relationship, $T(n)$ denotes the time required to solve the 1D closest pair problem for a set of n points.
- The term $2T(n/2)$ accounts for the two recursive closest pair computations performed on smaller subproblems.

Asymptotic analysis of Merge Sort

It involves understanding its time complexity, which is consistently $O(n \log n)$ in the worst, average, and best cases. Let's break down the analysis step by step.

Time Complexity Analysis:

- **Divide:** Dividing the array of size n takes $O(1)$ time.
- **Conquer:** The recursive calls on subproblems occur until each sublist contains only one element, resulting in $O(\log n)$ levels of recursion.
- **Combine (Merge):** Merging two sorted sublists of size $n/2$ takes $O(n)$ time.

Overall Time Complexity of Merge Sort can be expressed using following **recurrence relation** (discussed in the later)

$$T(n) = 2T(n/2) + O(n).$$

Binary Search

Binary search operates by iteratively dividing the search range in half, progressively narrowing down the possibilities at each step.

We can denote the time taken for a dataset of size 'n' as $T(n)$, where $T(n/2)$ accounts for the time spent searching within one-half of the dataset, and $O(1)$ represents the constant time required for comparisons and calculations.

This can be expressed using the recurrence relation:

$$T(n) = T(n/2) + O(1)$$

Recurrences

Recurrences in the context of computer science and mathematics refer to mathematical relationships or equations that describe the recurrence of a function or sequence in terms of its previous values. These equations express a recursive relationship, defining the value of a function for a certain input in terms of its values for smaller inputs. These are often used to analyze the time complexity of recursive algorithms.

Key Components of Recurrences:

1. **Base Case:** A recurrence relation typically includes a base case, which represents the starting point or condition where the recurrence stops. The solution for the base case is usually provided explicitly.
2. **Recursive Case:** The recurrence relation defines how to compute the value of the function for a given input based on the values of the function for smaller inputs. This is often expressed as a formula or equation involving the function's previous values.
3. **Initial Conditions:** Along with the base case, initial conditions may be specified to determine the values of the function for small input sizes where the recurrence relation alone might not apply.

Consider the Fibonacci sequence, where $F(n)$ represents the n^{th} Fibonacci number: $F(n) = F(n-1) + F(n-2)$ with base cases: $F(0) = 0, F(1) = 1$. In this example, the recurrence relation expresses the n^{th} Fibonacci number as the sum of the two preceding Fibonacci numbers. The base cases specify the starting values for $F(0)$ and $F(1)$.

Importance in Algorithm Analysis:

Recurrence relations are fundamental in the analysis of recursive algorithms. The time complexity of many recursive algorithms can be expressed through recurrence relations. Understanding and solving these recurrences help in predicting and analyzing the efficiency of algorithms, providing insights into their behavior as the input size grows. The recurrences play a crucial role in modeling and analyzing the recursive behavior of functions and algorithms, providing a powerful tool for expressing and understanding repetitive patterns in mathematical and computational contexts.

Recurrences

Example Factorial Function

Consider the factorial function $n!$, defined as the product of all positive integers up to n . The recurrence relation for factorial is as follows:

$$n! = n \times (n - 1)!$$

With the base case $0! = 1$. This recurrence relation expresses the factorial of n in terms of the factorial of $(n - 1)$. The solution to this recurrence is $n! = n \times (n - 1)!$, and it is used in many recursive algorithms.

Example Fibonacci Sequence

The Fibonacci sequence is defined as follows:

$$F(n) = F(n - 1) + F(n - 2)$$

With base cases $F(0) = 0$ and $F(1) = 1$. This recurrence relation expresses the n^{th} Fibonacci number in terms of the two preceding Fibonacci numbers. Solving this recurrence gives the well-known Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, ...

Example Merge Sort

Consider the recurrence relation for the time complexity of the Merge Sort algorithm:

$$T(n) = 2T\left(\frac{n}{2}\right) + n.$$

This recurrence expresses the time complexity $T(n)$ of sorting an array of size n in terms of sorting two halves of size $\frac{n}{2}$ and merging them. The solution to this recurrence is $T(n) = O(n \log n)$, making Merge Sort an efficient sorting algorithm.

Example Binary Search

The recurrence relation for the time complexity of Binary Search can be expressed as:

$$T(n) = T\left(\frac{n}{2}\right) + 1.$$

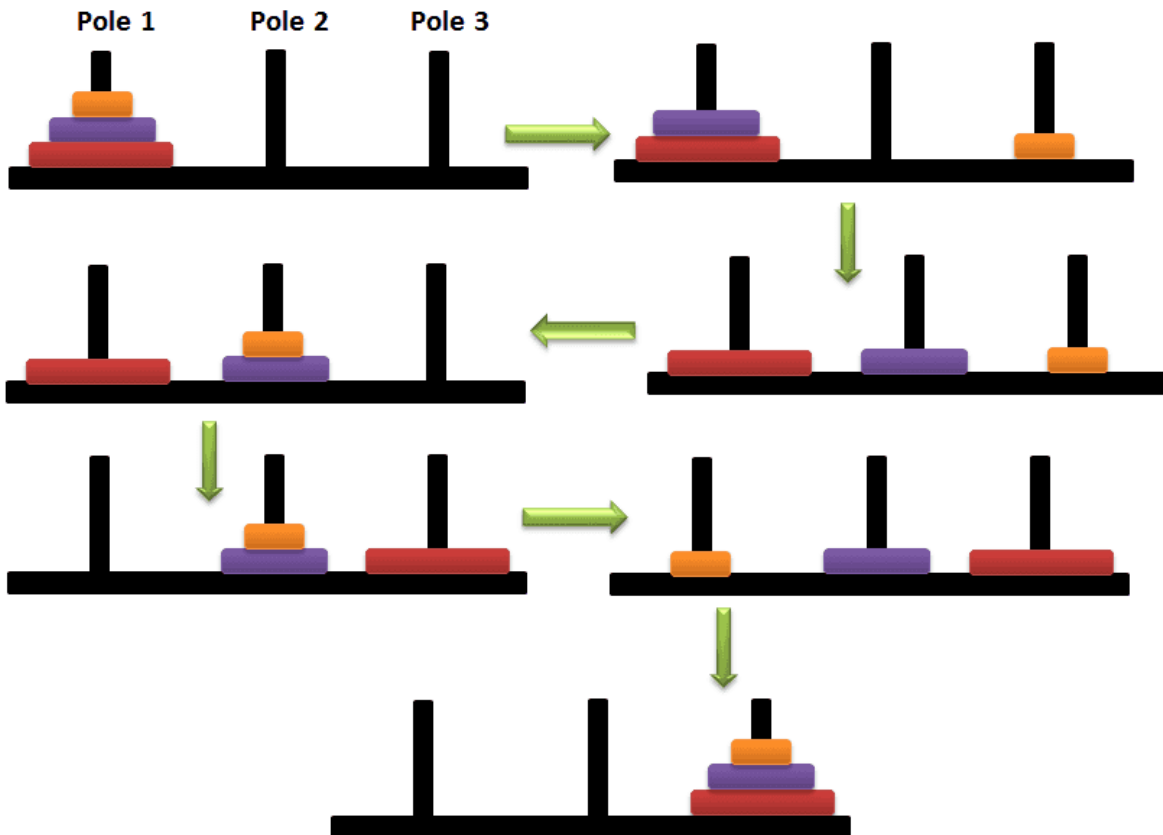
Recurrences

Example Towers of Hanoi

The recurrence relation for the number of moves required to solve the Towers of Hanoi problem with n disks is:

$$T(n) = 2T(n - 1) + 1.$$

With the base case $T(1) = 1$. This recurrence expresses the number of moves needed to solve the problem for n disks in terms of solving it for $n - 1$ disks twice and adding one additional move. The solution is $T(n) = 2^n - 1$.



Detailed in Stack
Chapter

Recurrences

There are several methods for solving recurrence relations, such as:

- **Substitution Method:** Guessing a solution and then proving it using mathematical induction.
- **Recurrence Tree Method:** Drawing a tree to represent the recursive calls and summing up the costs.
- **Master Theorem:** A specialized method for recurrence relations that fit a specific form.

Technicalities in recurrences :-

- ① In practice, we neglect certain technical details, when we state and solve recurrences.
- ② For example, if we call Merge-Sort on n -elements when n is odd, we end-up with subproblem of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. Neither size is actually $(n/2)$, because $n/2$ is not an integer when n is odd.
- ③ Technically, the recurrence describing the worst-case running time of Merge-Sort is really:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- ④ Boundary conditions represent another class of details that we typically ignore.
- ⑤ Since the running-time of an algorithm on a constant-sized input is a constant.

$$\boxed{T(n) = \Theta(1)} \text{ for sufficiently small } n.$$

⑤ We generally omit statements of the boundary conditions of recurrences and assume that $T(n)$ is constant for small n for convenience.

⑥ $T(n) = 2T(n/2) + O(n)$

↳ When we state and solve recurrences, we often omit floors^①, ceilings^②, and boundary conditions.

⑦ "They usually do not matter, but you should know when they do"

Master Theorem

The master method is a formula for solving recurrence relations of the form:

$$T(n) = aT(n/b) + f(n),$$

where, n = size of input

a = number of sub-problems in the recursion

n/b = size of each sub-problem.

All sub-problems are assumed to have the same size. $f(n)$ = cost of the work done outside the recursive call, which includes the cost of dividing the problem and the cost of merging the solutions

Here, $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.

An asymptotically positive function means that for a sufficiently large value of n , we have $f(n) > 0$.

Master Theorem

Master Theorem

If $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an asymptotically positive function, then the time complexity of a recursive relation is given by

$$T(n) = aT(n/b) + f(n)$$

where, $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} * \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$, then $T(n) = \Theta(f(n))$.

$\epsilon > 0$ is a constant.